

# Mil: Cost-guided Minimum Makespan Scheduling for Applications of Multiple LLMs

Jingzhi Fang · Yanyan Shen · Yue Wang · Lei Chen

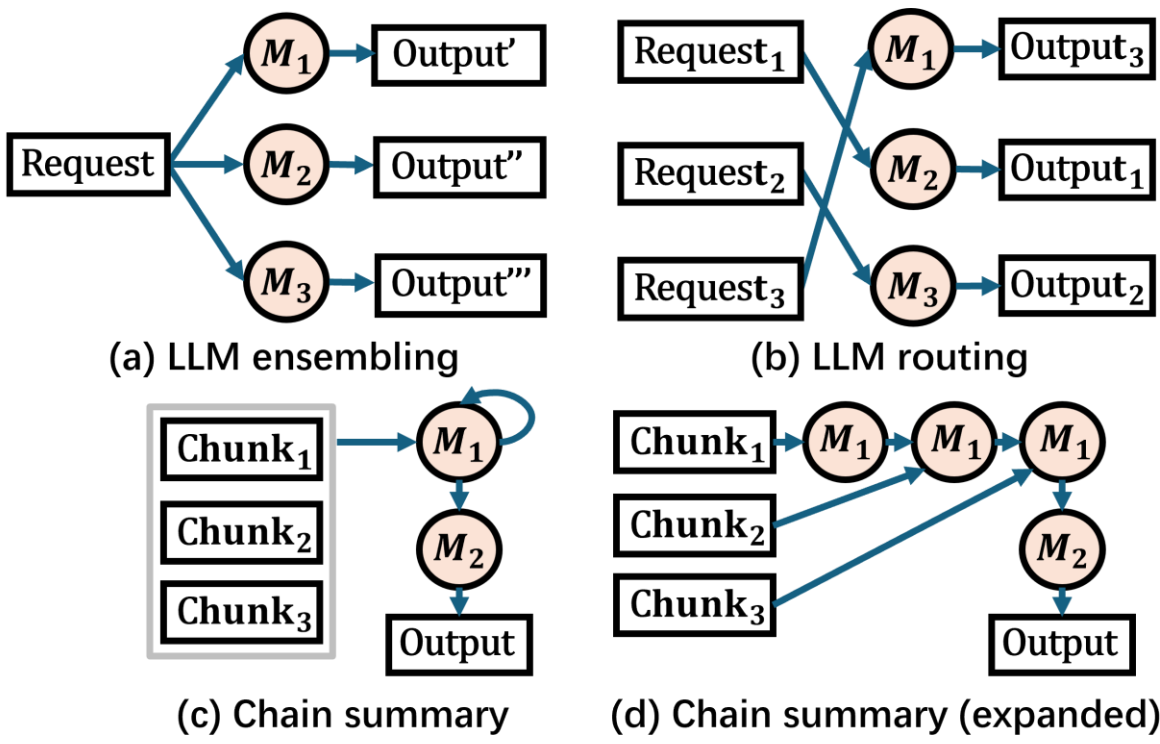
HKUST · Shanghai Jiao Tong University · Shenzhen Institute of Computing Sciences · HKUST(GZ)



Goal: minimize offline inference makespan for multi-LLM applications.

## 01 PROBLEM

### Multi-LLM applications



Computation graphs: nodes are LLMs; edges carry data.

### Offline scheduling jointly decides

INPUT: application graph + prepared requests + candidate execution plans + N GPUs

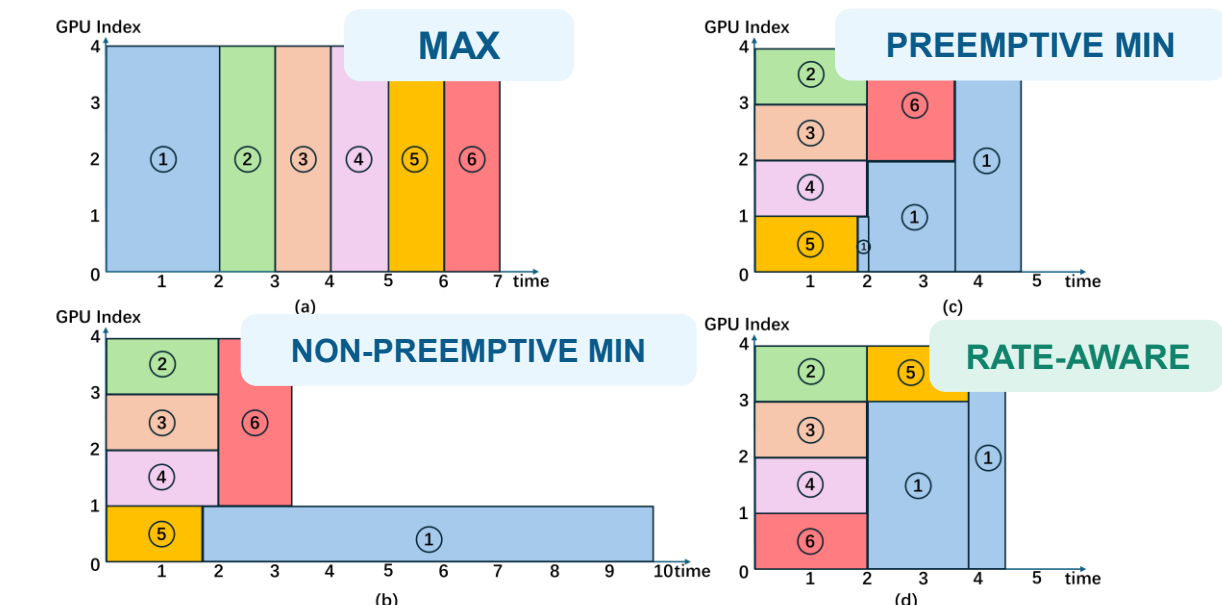
- 1 GPU allocation: how many GPUs for each model
- 2 Execution plan: parallelism strategy for each model
- 3 Model orchestration: timing, overlap, preemption

**MLAS objective**  $\min (\max_i C_i)$   
 $C_i$  is the completion time of model  $M_i$ .

## Why simple rules and prior art miss

Example: ensembling six LLMs on four GPUs

Data parallelism only · rates fixed during inference ·  $p_1$  linear in GPU count ·  $p_2 \sim p_6$  concave ( $p$ : the processing rate of model  $i$ )



MAX: Runs one LLM at a time with all GPUs; MIN: Uses minimum-GPU plans to maximize concurrency

**Key insight**  
The best allocation depends on each LLM's processing function  $p_i$ —not only on concurrency or GPU occupancy.

• Unknown output lengths make future batching states—and therefore processing rates—unknown; predicting output length for each request adds runtime cost.

• Rates change with the remaining requests; dependency-constrained models may overlap through model-level pipelines; model preemption is allowed.

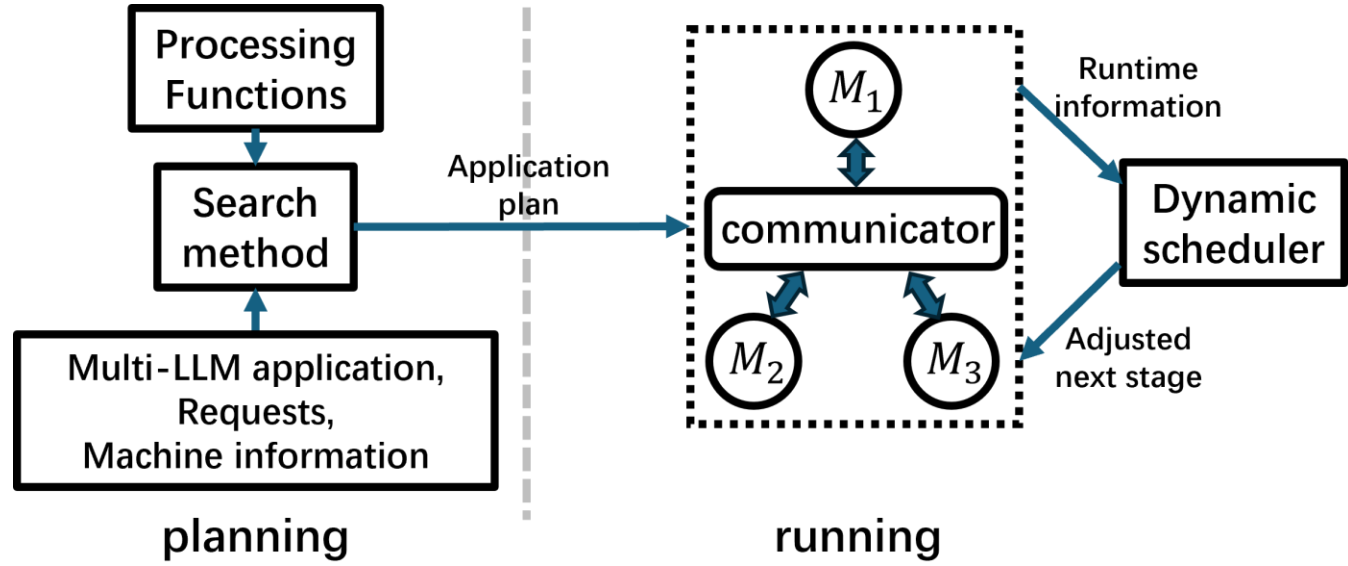
• Fixed/phase-rate schedulers do not fit MLAS; a direct MILP becomes extremely complex with MLAS's processing functions, preemption and pipeline overlap; generic heuristics ignore workload and rate differences.

### MLAS is NP-hard

Minimum-makespan scheduling for preemptive malleable jobs without precedence constraints is a strongly NP-hard special case.

## 02 MIL

### Mil at a glance



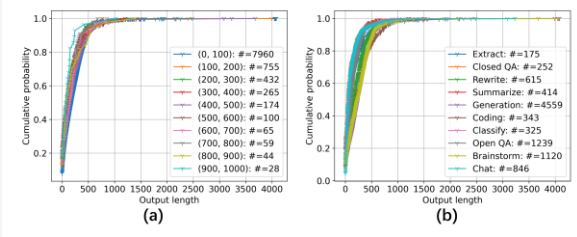
Offline planning builds the schedule; Mil adjusts it at runtime when actual model completion orders differ.

## 1 Processing-function estimation

**Key insight: sample output lengths for the entire request set, then simulate its continuous-batching trajectory—without training a per-request predictor.**

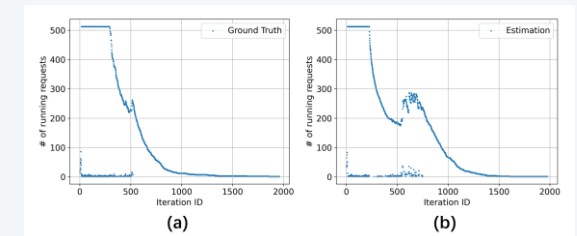
### 1 SAMPLE OUTPUT LENGTHS

To estimate the inference process for a model given a set of requests rather than predict the request latency, it is sufficient to know how the output length distribution. For normal unconstrained prompts, distributions are similar across input lengths and request categories.



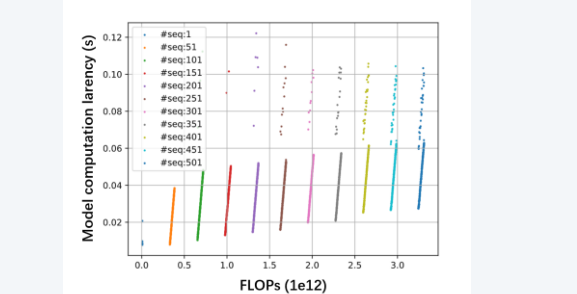
### 2 SIMULATE CONTINUOUS BATCHING

Request-ready times + scheduling policy + KV cache → batch and sequence lengths at every generation iteration.



### 3 ESTIMATE EACH ITERATION

Profile linear latency functions and estimate  $t_k = t_{\text{comp}} + t_{\text{prep}} + t_{\text{stamp}}$  for every simulated iteration.



Inputs: unfinished requests  $I_j^t$  · ready times  $H_j^t$  · execution plan  $P$  · interval  $\Delta t$

$$p_i(I_j^t, H_j^t, P, \Delta t) = \sum \text{FLOPs}_k / \Delta t$$

Model loading cost and communication setup cost are added whenever a model starts or changes its execution plan.

## 2 Greedy schedule search

Maximize total processing rate at each time point, while starting long-running ready LLMs early.

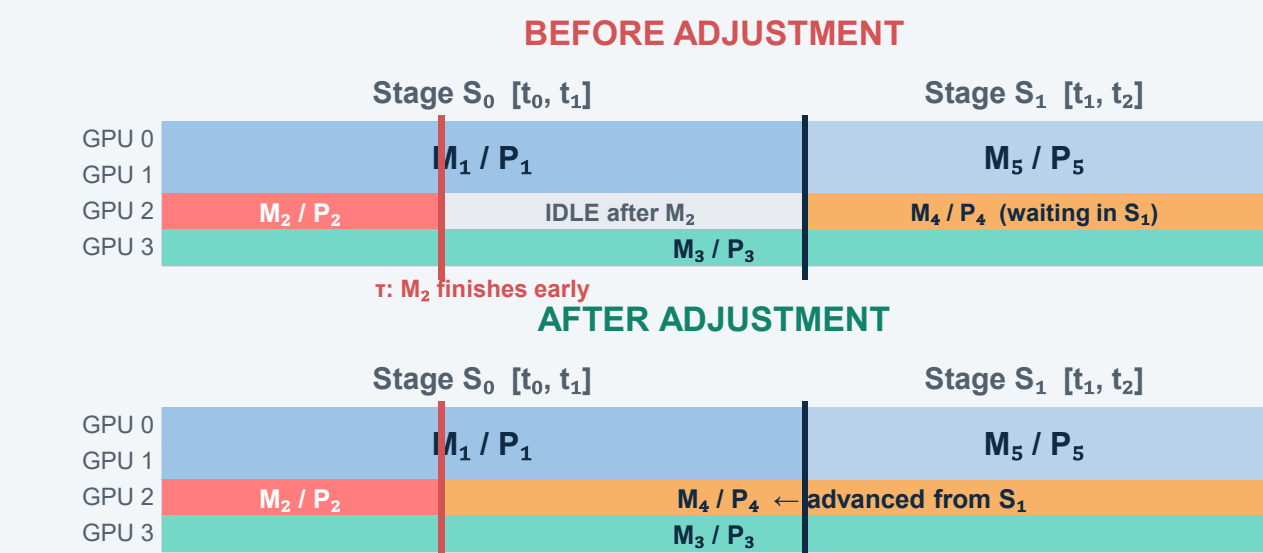
- 1 COMPUTE READY MODELS: Find models whose precedence constraints are satisfied in the current tentative interval.
- 2 START LONG JOBS EARLY: Sort by min-GPU completion time; move the longest into the candidate set, and return it if Step 3 selects no plan.
- 3 SELECT ONE BEST UPDATE: Enumerate feasible plan updates; maximize  $\Delta$  processing rate /  $\Delta$  GPUs.
- 4 FINALIZE ONE INTERVAL: End at the earliest predicted completion and record model-plan assignments.

**PLAN UPDATE** Choose the next best feasible plan update iteratively (repeat Steps 2 and 3 iteratively).  
**READY-SET REFRESH** All ready candidates have plans → refresh the ready set → repeat search for the current interval.  
**TIME ADVANCE** GPUs full / no improving update → finalize interval → advance time → update remaining requests.

Bound under simplified assumptions (a model's rate depends only on remaining request count; its best rate is concave in GPU count):  $t_i(A) \leq (1/\alpha + \beta)N \cdot t_i(A')$   
The bound tightens with stronger GPU scaling and smaller preemption cost.

## 3 Runtime schedule adjustment

If  $M_2$  finishes before  $M_1$ , advance a ready model-plan pair from a later planned interval onto the released GPUs to reduce GPU idleness.



## 03 EVALUATION

**3.4×** maximum end-to-end speedup  
Up to 4.0× in the scalability test

Metric: end-to-end time = schedule search (extra time) + inference

8× A100-80GB · vLLM · single node

### Applications

Ensembling · Routing · Chain summary · Mixed

### 8 baselines in three categories

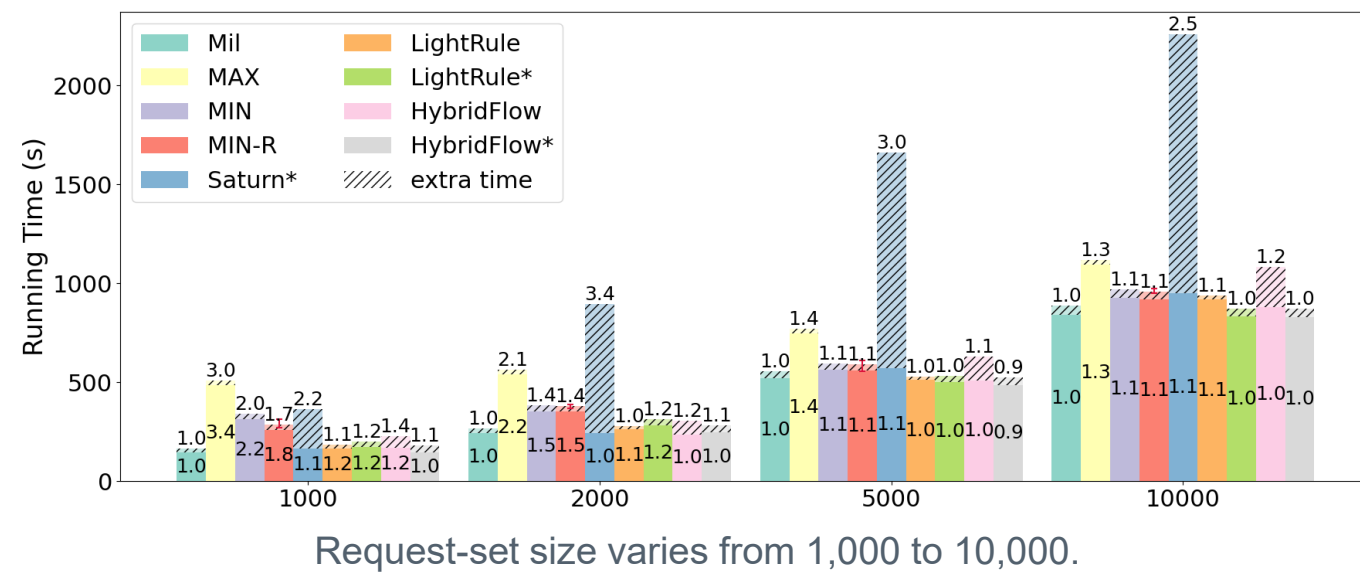
Heuristics: MAX · MIN/MIN-R · LightRule / LightRule\* (list scheduling+greedy allocation)

MILP: Saturn\*

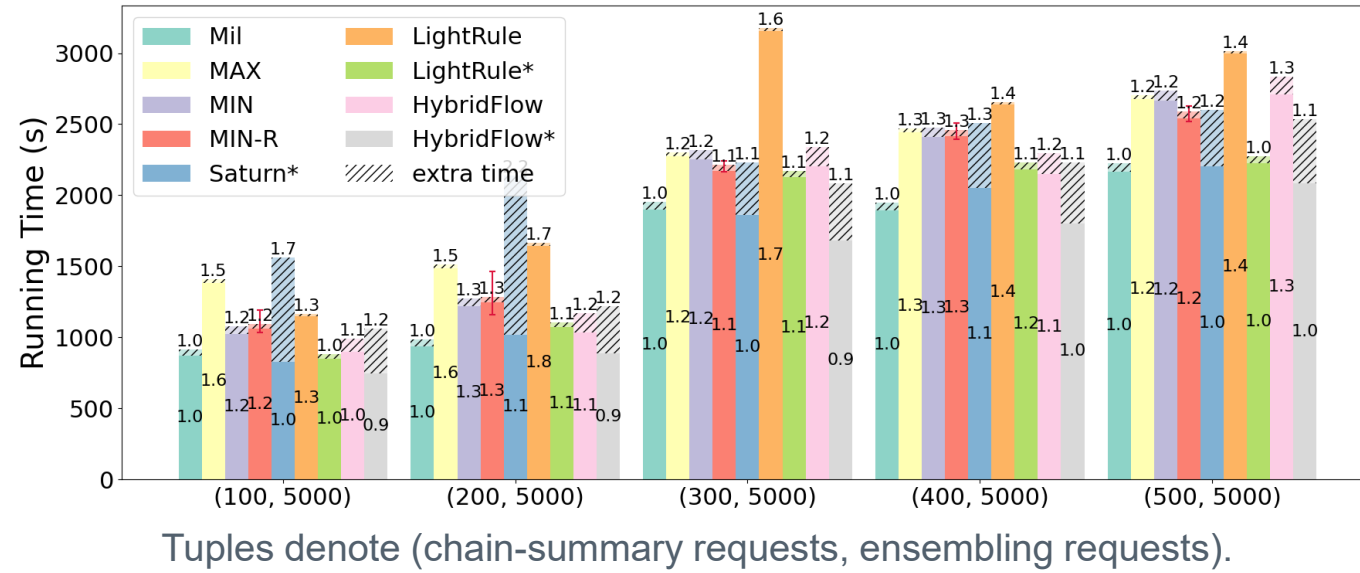
Enumeration: HybridFlow · HybridFlow\*

\* adapted/variant implementation · lower end-to-end time is better

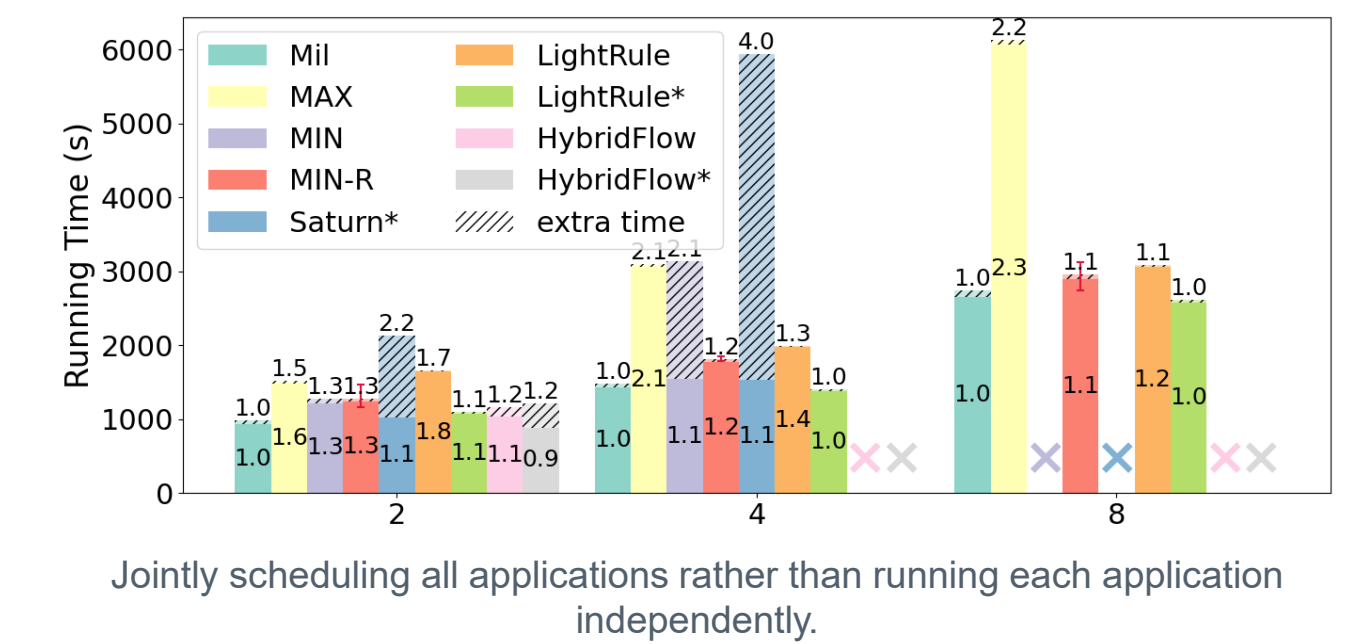
### Single application · LLM ensembling



### Mixed applications · chain summary + ensembling



### Scalability · 2, 4, and 8 applications



Jointly scheduling all applications rather than running each application independently.

### Where the gains come from

- Processing functions capture how rate changes with the evolving remaining request set and execution plan.
- Cost-guided greedy search jointly chooses GPU allocation, parallel plans, and timing.
- Runtime adjustment fills released GPUs when actual completion order differs.

### TAKEAWAY

Mil makes offline multi-LLM inference workload-aware: estimate processing rates, plan a good schedule, then adapt it to real completions.



github.com/puddingfjz/Mil